Gráfica Multimedia

Producción

Lourdes Marco · lmarco@esdmadrid.es

ESDM

JavaScript

Interpolación de cadena

Interpolación de cadenas

Podemos usar comillas invertidas `` para crear cadenas de texto.

El formato **${variable}** nos permite insertar un valor dinámico en la cadena, sin necesidad de tener que estar concatenando todo.

Otra utilidad interesante en la interpolación de cadenas es la posibilidad de crear **cadenas multilínea** muy fácilmente.

JS

```
let nombre = `Arya Stark`;
let p =  document.querySelector("#parrafo");

p.innerHTML += `Hola, soy ${nombre}
  y esto es un ejemplo
  de cómo escribir cadenas
  multilínea cómodamente.`;
```

CSS

```
p{
    white-space: pre-line;
}
```

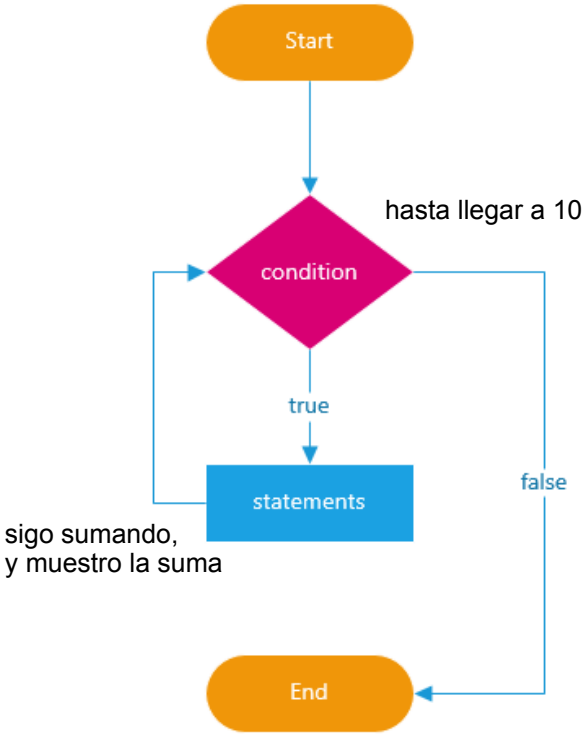
JavaScript

Bucles

Repetir acciones

Cuando estamos escribiendo programas, es muy común que necesitemos repetir acciones.

Por ejemplo, mostrar varios elementos de una lista o simplemente ir mostrando un contador de números del 1 al 10.

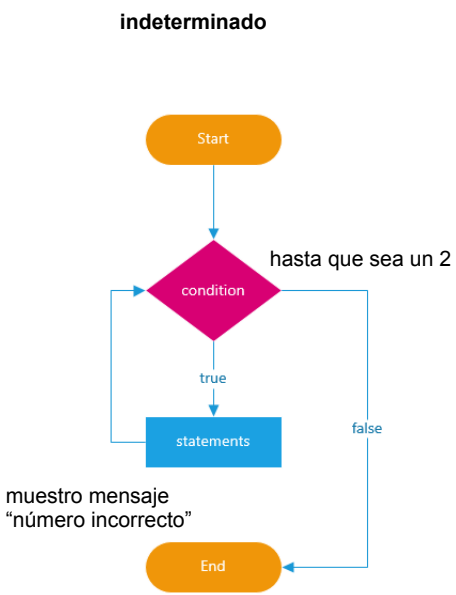


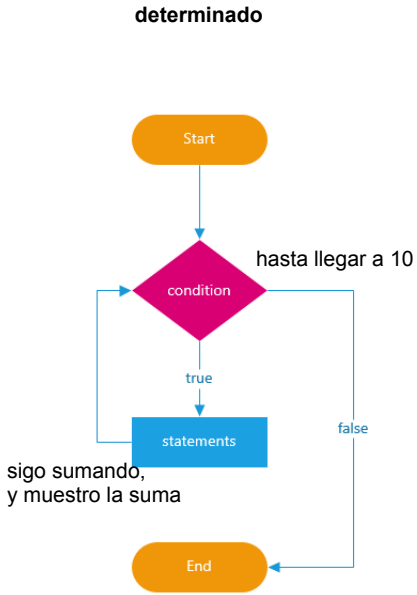
Bucles o iteradores

El bucle o el iterador es una **estructura de control** que nos permite repetir un conjunto de sentencias un **número determinado** o **indeterminado de veces** en base a una condición.
Las condiciones devuelven true o false al ser evaluadas. El bucle continuará ejecutándose hasta que la condición devuelva false.

- Decimos que es **indeterminado** cuando no sabemos cuantas veces se va a repetir. (Por ejemplo, hasta que el usuario introduzca el número 2).

- Decimos que es **determinado** cuando indicamos el número de veces que se repite. (Por ejemplo, un contador de 1 a 10).



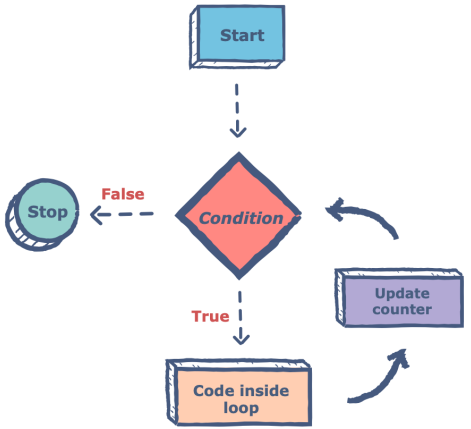


Bucles o iteradores

Hay muchos tipos diferentes de bucles, pero esencialmente, todos hacen lo mismo: **repiten una acción varias veces.**

Vamos a ver los más básicos, similares en otros lenguajes de programación, que son:

- **while**
- **do..while**
- **for**



While

While repite un bloque de sentencias mientras la expresión a evaluar devuelva true.

**** Es importante evitar los bucles infinitos!** Si olvidamos incrementar la variable, el bucle nunca termina y la pestaña del navegador se cuelga.

```
while(condición) {
   Sentencia 1;
   Sentencia 2;
   Sentencia N;
}
```

JS

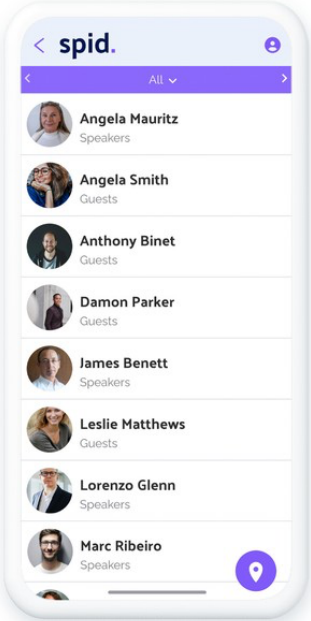
```
let limite = 10;
let contador = 1;
let p =  document.querySelector("#bucle");

while (contador <= limite) {
   p.innerHTML += `Vuelta número ${contador} <br>`;
   contador++;
}
```

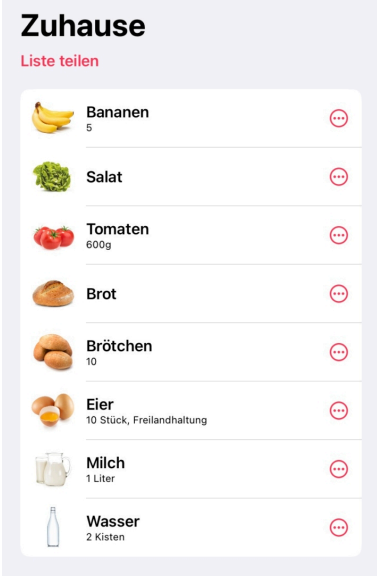
Ejemplos de bucles en una GUI

Siempre que tenemos que pintar por pantalla una **lista de elementos con la misma estructura** (contactos, publicaciones, categorías, etc…) es un buen momento para recurrir a un bucle.

Ya sea que estemos creando la lista, añadiendo un elemento, borrándolo, o modificándolo; **recorrer el total de elementos será una tarea habitual**.



Lista de usuarios



Lista de la compra

do…while

do while es una variante del bucle while, y funciona de manera similar.

La diferencia más notable se da en la ejecución. **El bucle do...while será ejecutado al menos una vez**, aunque la condición sea falsa, **porque el bloque de código se ejecuta antes de evaluar la condición**.

```
do {
   Sentencia 1;
   Sentencia 2;
   Sentencia N;
} while(condición);
```

JS

```
let num;

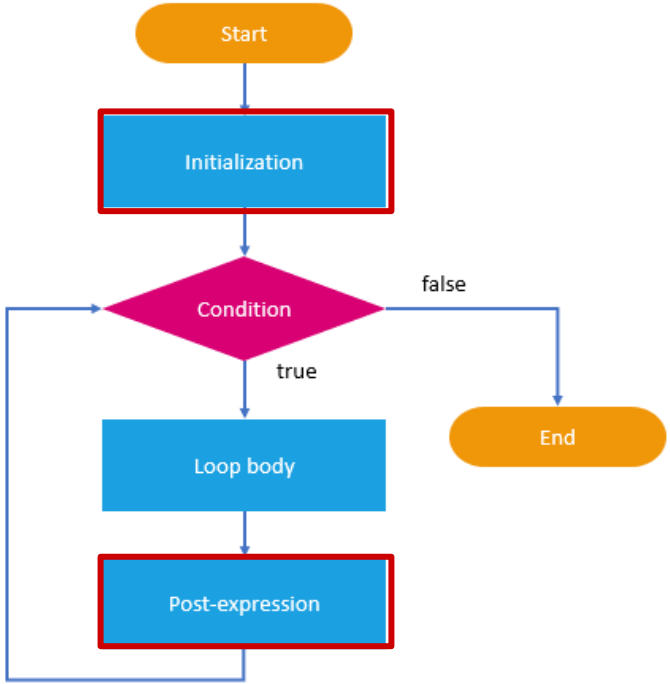
do{
    num = prompt(`Acierta el número`);
} while (num!=2) ;
```

for

El bucle for es algo más complejo que while, pero también es **la estructura más usada en JS**.
for permite crear un bucle con tres expresiones opcionales:

- **inicialización**
- **condición**
- **post-expresión**

```
for (inic.; condición; post-exp ) {
   Sentencia 1;
   Sentencia 2;
   Sentencia N;
};
```



for

- **inicialización. Es el valor en el que empieza el bucle**. Se ejecuta sólo una vez cuando se inicia el bucle.

- **condición. Es la expresión que se evalúa antes de cada iteración**. Las acciones dentro del bucle se ejecutan sólo cuando la condición es igual a true.

- **post-expresión. Es la expresión que se ejecuta después de cada iteración.** Se suele usar para actualizar el valor del contador.

JS

```
let i;
let p =  document.querySelector("#parrafo");

for (i = 1; i < 5; i++) {
    p.innerHTML += `Vuelta: ${i} <br>`;
}

p.innerHTML += `Total vueltas: ${i-1}`;
```

JavaScript

Calculadora simple para sumas

- Programa que pida números con prompt hasta que se pulse “cancelar”.
- Si no se introduce un número deberá indicarse con un alert() y seguir pidiendo números.
- Al salir con “cancelar” deberá indicarse la suma total de los números introducidos.



Posible solución

JS

```
let resultado = 0;
let num;

do {
    num = prompt("Introduce un número");
    if (num != "" && num !=null && Number(num)) {
        resultado = `${Number(resultado) + Number(num)}`;
    }
    if(!Number(num) && num !=null){
        alert(`Eso no es un número!!`);
    }
} while (num!=null);

if (resultado != 0){
    alert(`Total sumado: ${resultado}`);
}else{
    alert(`No has introducido ningún número`);
}
```

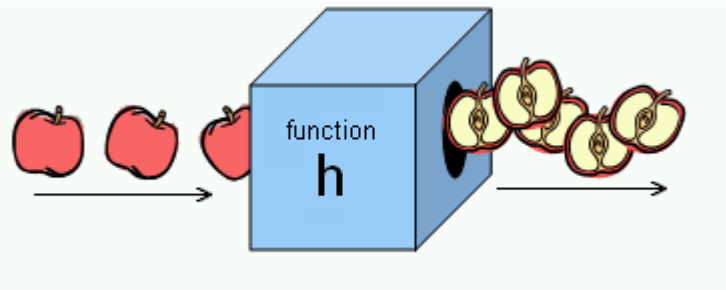
JavaScript

Funciones (forma tradicional)

Reutilizar código para tareas que se repiten

Una función es un **bloque de código** cuyo fin es **ejecutar una tarea** concreta. Podemos imaginarlas como una **máquina** que construímos para que haga cosas por nosotros, como partir en dos todas las manzanas que le echemos.

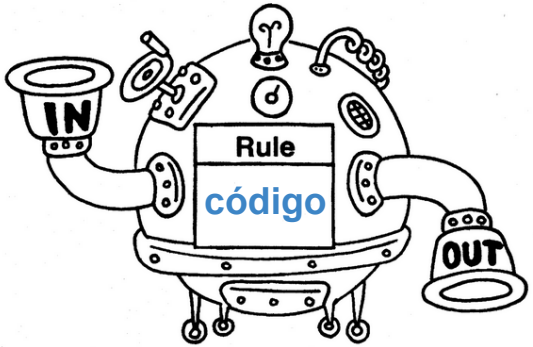
Si llevamos esto al código, podemos decir que **las funciones permiten reutilizar fragmentos de código.**
Se definen una vez y se usan muchas veces.



Declarar una función

Declarar una función equivaldría a construir la máquina, pero una cosa va a ser construirla y otra usarla.

La forma tradicional de declarar funciones en JS es usar la palabra clave function, seguida del nombre de la función, paréntesis y las llaves para el bloque de código a ejecutar.



JS

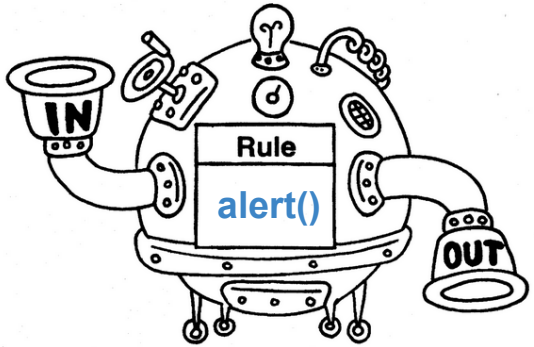
```
function nombreFun(){
    // código
}
```

Llamar a una función

Para que una función se ejecute, necesitamos **llamarla.** Si seguimos con el ejemplo de nuestra máquina, sería como darle al botón para encender la máquina.

Para llamar a una función **usamos el nombre con el que la hemos declarado**.

nombreFun (argumentos);



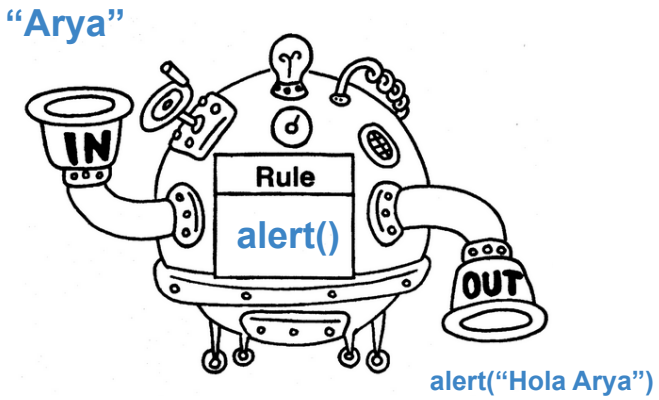
JS

```
// declaro la función
function showName(){
    alert("Arya Stark");
}

// llamo a la función
showName();
```

Parámetros en una función

Las funciones **pueden recibir datos** para llevar a cabo las tareas. Si recordamos el ejemplo de la máquina que corta manzanas, recibe manzanas completas y lo que obtenemos son medias manzanas.
Con el código es igual, **le pasamos datos (o parámetros)** a la función cuando la llamamos para realizar las tareas.



JS

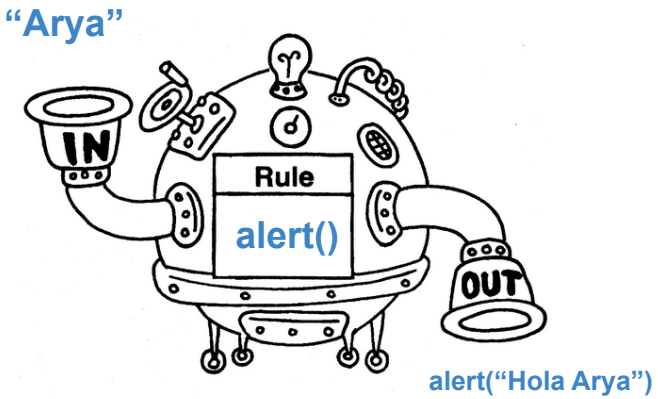
```
// declaro la función con el parámetro
function decirHola (nombre) {
    alert(`Hola ${nombre}`);
}

// llamo a la función
decirHola(“Arya”);
```

Múltiples parámetros en una función

A nuestra función podemos pasarle un dato o muchos.

Cuando sean muchos, los separamos con comas.



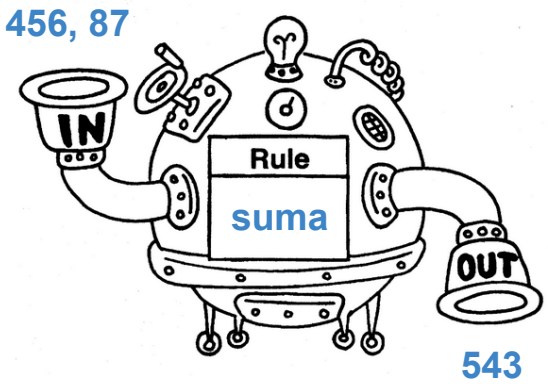
JS

```
// declaro la función con parámetros
function decirHola (nombre, apellido) {
    alert(`Hola ${nombre} ${apellido}`);
}

// llamo a la función
decirHola(“Arya”, “Stark”);
```

Return para devolver datos

Hasta ahora nuestras funciones han ejecutado alertas, y para este tipo de tarea no hemos necesitado que la máquina nos devuelva nada. Pero imaginemos que lo que necesitamos es que nuestra máquina **sume dos números y nos diga el resultado**. Como la suma se hace dentro de la función, vamos a necesitar decirle que nos devuelva el resultado, y para eso usamos **return**.



JS

```
// declaro la función con parámetros
function sumar (num1, num2) {
  return num1 + num2;
}

// llamo a la función y guardo el resultado
// en una variable
let total = sumar(456, 78);
```

ESDM

JavaScript

Funciones (forma actual)

Funciones por expresión

Con ES6 llegaron un par de variaciones a la hora de crear funciones que permitían **simplificar su definición.**

Por un lado, se empezaron a usar las **funciones de forma encapsulada en variables**. Lo que se conoce como **funciones por expresión**.

JS

```
// declaro la función
let sumar = function (num1, num2) {
  return num1 + num2;
}

// llamo a la función
let total = sumar(456, 78);
```

Arrow functions (funciones de flecha)

Con ES6 también llegaron las conocidas “Arrow functions”, o funciones de flecha, que permitían **simplificar al máximo la definición de una función** para evitarnos las llaves y las palabras function y return.

JS

```
// declaro la función
let sumar = (num1, num2) => num1 + num2;

// llamo a la función
let total = sumar(456, 78);
```

JavaScript

Saludando con funciones

- Programa que pida al usuario dos datos: un saludo y un nombre. Por ejemplo: “Hola” “Arya”

- Crea una **función (arrow function)** que reciba el saludo y el nombre, y que devuelva una frase que se muestre en un alert()



Posible solución

JS

```
// declaro la función
let saludar = (saludo, persona) => `${saludo} ${persona},¿Qué tal?`;

// pido los datos al usuario
let s = prompt("Introduce un saludo");
let p = prompt("Introduce un nombre");

// llamo a la función en el alert()
alert(saludar(s, p));
```

Parámetros por defecto

Hasta ahora hemos visto llamadas a funciones indicando todos los parámetros, pero ¿qué pasa si en la llamada a la función no le pasamos todos los parámetros? JS nos permite definir valores por defecto para estos casos.

JS

```
// declaro la función (y parámetro ape por defecto)
let decirHola = (nom, ape="Sin apellido") => alert(`Hola ${nom} ${ape}`);

decirHola("Arya");
decirHola("Arya", "Stark");
```